

Software Testing and Validation Lab

LAB CODE

8CS4-22

Submitted By:

Pranjal Khandelwal

22EARCS125

Under the supervision of

Dr.Krishan Kant Lavania

Professor



Session 2025-26

Department of Computer Science and Engineering

ARYA COLLEGE OF ENGINEERING & I.T.

SP-42, RIICO INDUSTRIAL AREA, KUKAS, RAJASTHAN



Course Introduction – Software Testing & Validation Lab – 8CS4-22

Course Title: Software Testing & Validation Lab

Course Code: 8CS4-22

Credits: 1 (L-T-P: 0-0-2)

Duration: 10 Sessions

Max. Marks: 100 (IA:60, ETE:40)

Target Semester: B.Tech 8th Semester (Computer Science & Engineering)

Course Overview:

The Software Testing & Validation Lab provides practical exposure to testing techniques, tools, and methodologies used in modern software development. Students learn to design test cases, perform coverage analysis, mutation testing, and web application testing using tools like **JaButi**, **JMeter**, **Selenium**, **Jumble**, and **EclEmma**.

The lab bridges the gap between theoretical testing concepts and real-world application validation.

Course Objectives

By the end of this course, students will be able to:

1. Understand fundamentals of software testing and validation.
2. Design effective test cases and analyze test coverage.
3. Apply testing tools such as JaButi, JMeter, Selenium, etc.
4. Perform mutation testing and coverage analysis.
5. Evaluate performance of real-world web applications.
6. Develop skills in automation testing and validation.



RAJASTHAN TECHNICAL UNIVERSITY, KOTA

Scheme & Syllabus

IV Year- VII Semester: B. Tech. (Computer Science & Engineering)

8CS4-22: Software Testing and Validation Lab

Credit: 1
OL+OT+2P

Max. Marks:100 (IA:60, ETE:40)

SN	List of Experiments		
1	a) Write a program that calculates the area and perimeter of the circle. And find the Coverage & Test Cases of that program using JaButi Tool. b) Write a program which read the first name and last name from console and matching with expected result by using JaBuTi. c) Write a program that takes three double numbers from the java console representing , respectively, the three coefficients a,b, and c of a quadratic equation. d) Write a program that reads commercial website URL from a url from file .you should expect that the URL starts with www and ends with .com. retrieve the name of the site and output it. For instance, if the user inputs www.yahoo.com, you should output yahoo. After that find the test cases and coverage using JaButi. e) Write a program for a calculator and find the test case and coverage and Def-use-graph. f) Write a program that reads two words representing passwords from the java console and outputs the number of character in the smaller of the two. For example, if the words are open and sesame, then the output should be 4, the length of the shorter word, open. And test this program using JaButi		
2	Analyse the performance of following website using JMeter.		
	Site	Website	Type
	Amazon	Amazon.com	shopping
	Flip kart	Flipkart.com	shopping
	Railway reservation	Irctc.co.in	Ticket booking site
	Train searching	Erail.in	Train searching
3	Calculate the mutation score of programs given in 1(a) to 1 (f) using jumble Tool.		
4	Calculate the coverage analysis of programs given in 1 (a) to 1 (f) using Eclemma Free open source Tool.		



5

Generate Test sequences and validate using Selenium tool for given websites below:

Site	Website	Type
Amazon	Amazon.com	shopping
Flip kart	Flipkart.com	shopping
Railway reservation	Irctc.co.in	Ticket booking site
Train searching	Erail.in	Train searching



Course Outcomes:

By the end of this course, students will be able to:

CO Code	Detailed Course Outcome Statement	Mapped Experiments	Bloom's Level	Keywords
CO1	Design, develop, and execute comprehensive test cases for software programs by applying testing techniques such as equivalence partitioning, boundary value analysis, and input validation to ensure correctness and reliability.	Exp 1(a-f)	Apply (L3)	Test cases, validation, correctness
CO2	Analyze and evaluate software programs using code coverage techniques such as statement, branch, and path coverage to measure the effectiveness and completeness of test cases.	Exp 1(a-f), Exp 4	Analyze (L4)	Coverage, branch, path
CO3	Apply mutation testing techniques to generate mutants, execute test cases, and compute mutation score to assess fault detection capability and improve test quality.	Exp 3	Analyze (L4)	Mutation testing, mutants
CO4	Evaluate the performance and scalability of real-world web applications by designing and executing load and stress tests, and analyzing performance metrics such as response time, throughput, and latency.	Exp 2	Analyze (L4)	Performance testing, JMeter
CO5	Design and implement automated test scripts using Selenium for validating web applications, including navigation, input handling, and result verification.	Exp 5	Apply, Create (L3-L6)	Automation, Selenium
CO6	Demonstrate proficiency in using modern software testing tools and frameworks to perform validation, analyze results, and ensure software quality in real-world scenarios.	All Experiments	Apply (L3)	Testing tools, QA



Assessment Plan

Component	Marks
Internal Assessment (IA-1) – Lab Files, Viva, Attendance, Mini Project	30
Internal Assessment (IA-2) – Lab Files, Viva, Attendance, Mini Project	30
External Exam (ETE) – Practical Test + Viva	40
Total	100



Program Outcomes (POs)

PO1: Engineering Knowledge

Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2: Problem Analysis

Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3: Design/Development of Solutions

Design solutions for complex engineering problems and design system components or processes that meet specified needs with appropriate consideration for public health and safety, and cultural, societal, and environmental factors.

PO4: Conduct Investigations of Complex Problems

Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of information to provide valid conclusions.

PO5: Modern Tool Usage

Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of limitations.

PO6: The Engineer and Society

Apply reasoning informed by contextual knowledge to assess societal, health, safety, legal, and cultural issues and the responsibilities relevant to professional engineering practice.

PO7: Environment and Sustainability

Understand the impact of professional engineering solutions in societal and environmental contexts and demonstrate knowledge of sustainable development.

PO8: Ethics

Apply ethical principles and commit to professional ethics and responsibilities and norms of engineering practice.

PO9: Individual and Team Work

Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.



PO10: Communication

Communicate effectively on complex engineering activities with the engineering community and society at large, including writing reports, making presentations, and giving/receiving clear instructions.

PO11: Project Management and Finance

Demonstrate knowledge and understanding of engineering and management principles and apply these to manage projects as a member or leader in multidisciplinary environments.

PO12: Life-long Learning

Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the context of technological change.



Program Educational Objectives (PEOs)

PEOs are broad statements that describe the career and professional accomplishments that the program is preparing its graduates to accomplish. These objectives were developed, refined and finalized using the inputs from various constituencies such as the academic experts, staff of Computer Engineering department, Alumni of the college working in field and various industries visiting the college for on campus placement beside the country basic need for human resources in the field of Computer engineering.

PEO I: The graduates of the Program will have solid foundation in the principles and practices of computer science, including mathematics, science and basic engineering.

PEO II: The graduates of the Program will have skills to function as members of multidisciplinary teams and to communicate effectively using modern tools.

PEO III: The graduates of the Program will be prepared for their careers in the software industry or pursue higher studies and continue to develop their professional knowledge.

PEO IV: The graduates of the program will practice the profession with ethics, integrity, leadership and social responsibility.

Program Specific Outcomes (PSO)

The department of computer science and engineering has framed the following Program Specific Outcomes.

PSO 1. Students are able to model, design and implement Software and hardware communication network systems using knowledge of modern tools and techniques to serve the society as professional computer engineers.

PSO 2. Students are able to develop software engineering system of varying complexity and Innovative products in emerging areas of Computer Engineering.

**CO-PO Mapping Matrix for 8CS4-22: Software Testing & Validation Lab**

CO Code	Course Outcome Statement	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12
CO1	Design, develop, and execute comprehensive test cases for software programs by applying testing techniques such as equivalence partitioning, boundary value analysis, and input validation to ensure correctness and reliability.	3	3	2	–	2	–	–	–	1	1	–	–
CO2	Analyze and evaluate software programs using code coverage techniques such as statement, branch, and path coverage to measure the effectiveness and completeness of test cases.	3	3	2	2	3	–	–	–	1	1	–	–
CO3	Apply mutation testing techniques to generate mutants, execute test cases, and compute mutation score to assess fault detection capability and improve test quality.	3	3	2	2	3	–	–	–	1	1	–	–
CO4	Evaluate the performance and scalability of real-world web applications by designing and	3	3	3	2	3	1	1	–	2	1	1	–



CO Code	Course Outcome Statement	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12
	executing load and stress tests, and analyzing performance metrics such as response time, throughput, and latency.												
CO5	Design and implement automated test scripts using Selenium for validating web applications, including navigation, input handling, and result verification.	3	2	3	2	3	-	1	-	2	2	-	1
CO6	Demonstrate proficiency in using modern software testing tools and frameworks to perform validation, analyze results, and ensure software quality in real-world scenarios.	3	3	3	2	3	1	1	1	3	2	2	2

PO Scale Legend:

3 = Strongly related, 2 = Moderately related, 1 = Slightly related, - = Not related

**CO-PSO Mapping Matrix for 8CS4-22: Software Testing & Validation Lab**

Course Outcome (CO)	PSO1	PSO2
CO1	2	3
CO2	2	3
CO3	2	3
CO4	2	3
CO5	2	3
CO6	2	3

PSO Scale Legend:

3 = Strongly related, 2 = Moderately related, 1 = Slightly related

PEO-PO/PSO Mapping Matrix for 8CS4-22: Software Testing & Validation Lab

PEO \ PO/PSO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PSO1 0	PSO1 1	PSO1 2	PSO2 1	PSO2 2
PEO I (Strong foundation in CS fundamentals)	3	2	2	2	2	–	–	–	1	1	–	2	2	3
PEO II (Teamwork & communication skills)	2	2	2	–	2	1	1	1	3	3	2	2	2	3
PEO III (Career readiness & higher studies)	3	3	3	2	3	1	1	1	2	2	2	3	2	3
PEO IV (Ethics, leadership & social responsibility)	2	2	2	1	2	3	3	3	3	2	2	2	2	3

**Lab Plan 8CS4-22: Software Testing & Validation Lab**

Exp. No.	Title of Experiment	Mapped CO	Mapped PO(s)	Mapped PEO(s)	Mapped PSO(s)	Bloom's Level
1 (Session-1)	Programs: Circle & Name Matching – Test case design using JaButi (1a, 1b)	CO1, CO2	PO1, PO2, PO5	PEO1	PSO2	Apply
1 (Session-2)	Programs: Quadratic, URL, Calculator, Password – Coverage & testing using JaButi (1c–1f)	CO1, CO2	PO1, PO2, PO5	PEO1, PEO2	PSO2	Apply, Analyze
2 (Session-3)	Performance testing setup & execution for Amazon, Flipkart using JMeter (2)	CO4	PO2, PO4, PO5	PEO2	PSO2	Analyze
2 (Session-4)	Performance testing analysis for IRCTC, Erail using JMeter (2)	CO4	PO2, PO4, PO5	PEO2	PSO2	Analyze
3 (Session-5)	Mutation testing of basic programs (1a–1c) using Jumble Tool (3)	CO3	PO2, PO4	PEO2	PSO2	Analyze
3 (Session-6)	Mutation testing of remaining programs (1d–1f) using Jumble Tool (3)	CO3	PO2, PO4	PEO2	PSO2	Analyze
4 (Session-7)	Coverage analysis of basic programs (1a–1c) using EclEmma (4)	CO2	PO2, PO5	PEO2	PSO2	Analyze
4 (Session-8)	Coverage analysis of remaining programs (1d–1f) using EclEmma (4)	CO2	PO2, PO5	PEO2	PSO2	Analyze
5 (Session-9)	Selenium automation setup and testing for Amazon & Flipkart (5)	CO5, CO6	PO3, PO5, PO9	PEO3	PSO2	Apply
5 (Session-10)	Selenium automation and validation for IRCTC & Erail (5)	CO5, CO6	PO3, PO5, PO9	PEO3, PEO4	PSO2	Apply, Create



Lab Rules and Guidelines

Do's

1. Switch off or silence your mobile phone before entering the lab.
2. Enter the lab with complete **source code and necessary data**.
3. Check that all peripherals (keyboard, mouse, etc.) are working before starting.
4. Inform the Lab Incharge immediately if you face system issues or software problems.
5. Arrange your seat and peripherals properly before leaving.
6. Shut down the system correctly after use.
7. Keep bags in the designated racks outside the lab.
8. Maintain punctuality—enter and leave the lab on time.
9. Maintain discipline and decorum at all times.
10. Use lab hours effectively for the assigned experiment.
11. Get any CD/Pen drive verified by the Lab Incharge before use.

Don'ts

1. Do not bring unauthorized storage devices (Pen drives, etc.) without permission.
2. Do not mishandle or damage the system.
3. Do not leave systems running idle for long periods.
4. Do not bring unnecessary external materials into the lab.
5. Avoid making noise or disturbing others.
6. Do not use mobile phones inside the lab unless absolutely necessary.
7. Do not enter the lab without permission from the Lab Incharge.
8. Do not litter; keep the lab clean.
9. Do not delete or modify system files.
10. Do not take any lab equipment outside the lab.

Your cooperation is essential for the smooth and efficient functioning of the laboratory.



Instructions for Students

Before Entering the Lab

- All students must prepare the theoretical concepts related to the upcoming experiment.
- Students must bring their **practical file** and **lab notebook** to every session.
- Assignments given in previous labs must be completed and properly written in the practical file.
- Printed diagrams (if required) should be neatly pasted in the lab file.
- Students who fail to follow these instructions may **not be allowed to enter the lab**.

While Working in the Lab

- Follow the experimental schedule as instructed by the **Lab Incharge**.
- Ensure that previously completed programs are verified and signed by the instructor.
- Get the output of the current program checked and approved in your lab notebook.
- Each student must work only on their **assigned computer system**.
- Handle all lab equipment and accessories responsibly.
- Stay focused on the assigned experiment; playing games or unrelated activities is strictly prohibited.
- Any student found attempting to take lab equipment outside will face **strict disciplinary action**.

Experiment – 01(a)

Date of Performance: 09.01.2026	Date of Evaluation: 10.01.2026
---------------------------------	--------------------------------

Objective : Write a program that calculates the area and perimeter of the circle. And find the Coverage & Test Cases of that program using JaButi Tool.

Program Code: AreaOfCircle.java

```
import java.util.Scanner;
import java.lang.Math;
public class AreaOfCircle
{
    public static void main(String[] args)
    {
        int option;
        double radius, circumference, diameter, area;
        //object of the Scanner class
        Scanner sc=new Scanner (System.in);
        //options available
        System.out.println("1. If the radius is known");
        System.out.println("2. If the diameter is known");
        System.out.println("3. If the circumference is known");
        System.out.print("Enter your choice: ");
        //taking an option as input from the user
        option=sc.nextInt();
        switch(option)
        {
            //Case statements
            case 1:
                System.out.print("Enter the radius of the circle: ");
                radius=sc.nextDouble();
                area=(Math.PI*(radius*radius));
                System.out.print("The area of the circle is: "+area);
                break;
            case 2:
                System.out.print("Enter the diameter of the circle: ");
                diameter=sc.nextDouble();
```



```
area=Math.PI*(diameter*diameter)/4;  
System.out.print("The area of the circle is: "+area);  
break;
```

```
case 3:  
System.out.print("Enter the circumference of the circle: ");  
circumference=sc.nextDouble();  
area=(circumference*circumference)/(4*Math.PI);  
System.out.print("The area of the circle is: "+area);  
break;  
//default case statement executes when an invalid choice is entered  
default:System.out.println("invalid choice!");  
}  
}  
}
```

Compile Java Program:>_javac AreaOfCircle.java

It will create a new file AreaOfCircle.class

// BYTE CODE

Now assign this ByteCode to JaBUTi Tool for Coverage Testing by following steps

1. JaBUTi Functionality and Graphical Interface

JaBUTi implements a subset of the functionalities provided by xSuds, a complete tool suite for C and C++ programs. This section describes the operations that can be performed by JaBUTi. The graphical interface allows the beginner to explore and learn the concepts of control-flow and data-flow testing. Moreover, it provides a better way to visualize which part of the classes under testing are already covered and which are not. A general view of the JaBUTi graphical interface, including its menus, is presented in Figure.

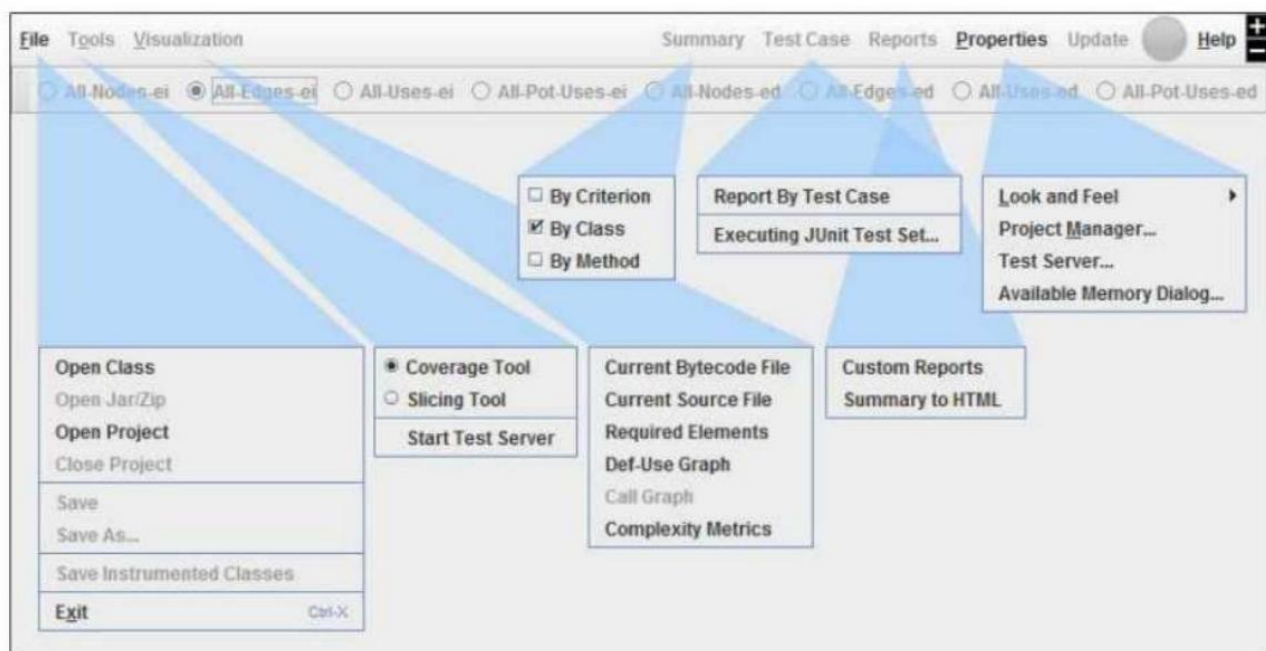


Figure 1. A brief description of each option on each menu

2. How to Create a Testing Project in JaBUTi

The testing activity requires the creation of a testing project. A testing project is characterized by a file storing the necessary information about (i) the base class file, (ii) the complete set of classes required by the base class, (iii) the set of classes to be instrumented (tested), and (iv) the set of classes that are not under testing. Additional information, such as the CLASSPATH environment variable necessary to run the base class is also stored in the project file, whose extension is .jbt. During the execution of any .class file that belongs to the set of classes under testing of a

given project, dynamic control-flow information (execution trace) is collected and saved in a separate file that has the same name of the project file but with a different extension (.trc).

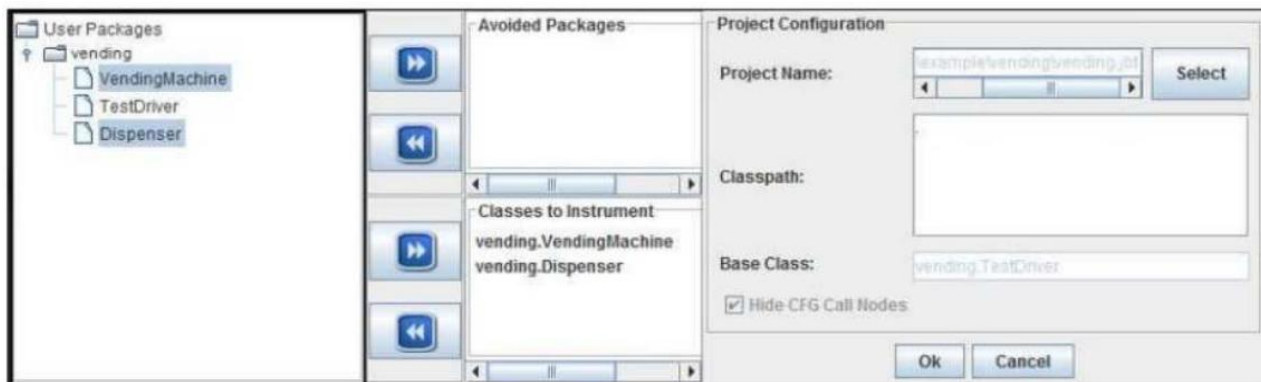
To create a new project, the first step is to select a base .class file from File → Open Class menu. A dialog window, as illustrated in following Figure.

From this dialog window, the tester selects the directory where the base class file is



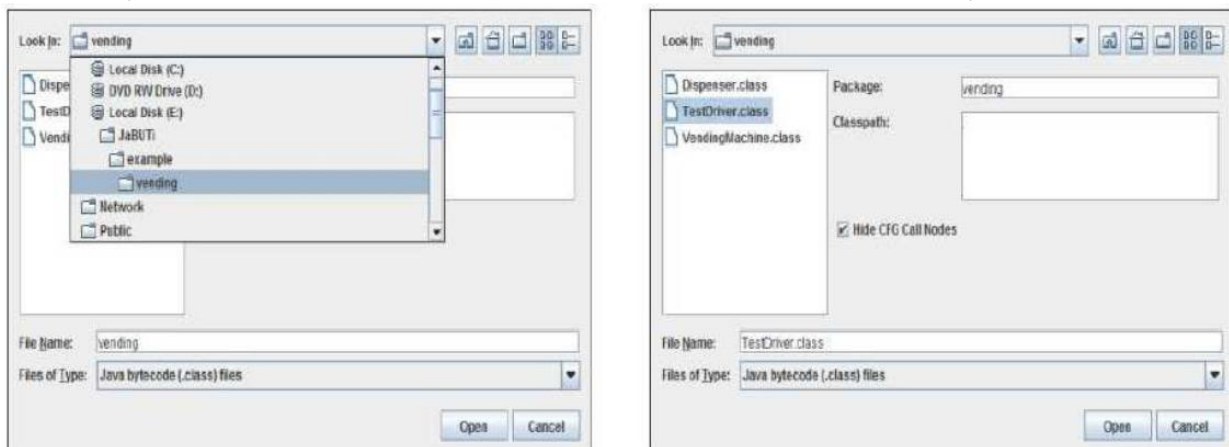
located and then select the base class file itself. Once the base class file is selected the tool automatically identifies the package that it belongs (if any) and fills out the Package field with the package's name. The Classpath should contains only the path necessary to run the selected base class.

By clicking the Open button the Project Manager window, will be displayed.



From the selected base class file the tool identifies the complete set of system and non-system class files necessary to execute class file

Currently, JaBUTi does not allows the instrumentation of system class files.



Therefore, the complete set of non-system class files (user's classes) relate to selected class can be instrumented. From the Project Manager window (left side) the user can select the class files that will be tested. At least one class file must be selected.

Moreover, the tester must give a name to the project being created by clicking on the Select button.

By clicking on the Ok button, JaBUTi creates a new project, constructs the DUG for each

method of each class under testing, derives the complete set of testing requirements for each criterion, calculates the weight of each testing requirement, and presents the bytecode of a given class under testing. By creating a project, the tester does not

need to go through this entire process again if he/she intends to test the same set of class files. Next time, he/she can just go to File → Open Project menu option and reload a previously saved project.

Once the project is created, the tester just need to open it and the tool automatically detects the classes that compose it.

JaBUTi can be used for analyzing the coverage of a given class file, for debugging the class file using the slice tool and for collecting static metrics information.

JaBUTi also allows the visualization of the Def-Use Graph (DUG) of each method in a given class file as well as the visualization of the source code, when available.

Different kinds of testing reports can also be generated considering different levels of abstraction.

3. How to use JaBUTi as a Coverage Analysis Tool

By default, the tool displays the bytecode instead of the Java source code, since the source code may not be available. The user can use the Visualization menu to change the display to show the bytecode or source code, as well as the DUG (Depth Use Graph) of each method in the current class.

Figure 5 shows the **colored bytecode view** of the program in JaBUTi before executing test cases, where different colors indicate the **weight (importance) of testing requirements**. Higher weight (red color) means covering that part will increase overall coverage more.



Figure 5: Dispenser.dispenser method: bytecode prioritized w.r.t. All-Pri-Nodes criterion.

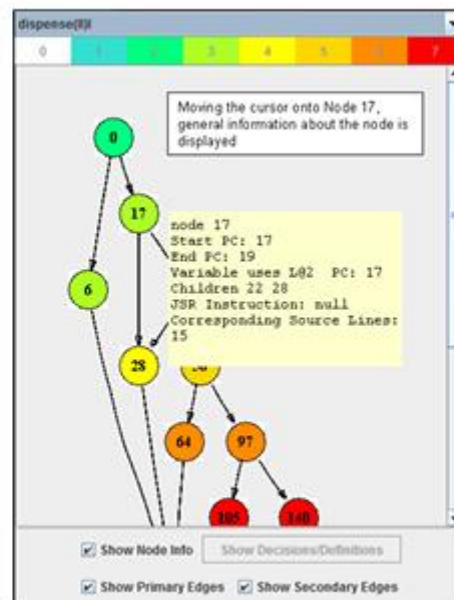


Figure 6: Dispenser.dispenser method: DUG prioritized w.r.t. All-Pri-Nodes criterion.

4. Highlighting of Testing Requirements in JaBUTi

JaBUTi highlights testing requirements using different colors based on the selected testing criterion such as All-Nodes, All-Edges, and All-Uses. For node-based criteria, each node is colored according to its weight, indicating its importance in improving coverage. In edge-based criteria, a two-layer approach is used where decision nodes are highlighted first, and clicking them shows their branches with respective weights. Higher weight indicates higher priority for testing. Nodes or edges with zero weight are shown in white, meaning they are already covered. Similarly, for def-use criteria, definitions are shown first, and selecting them

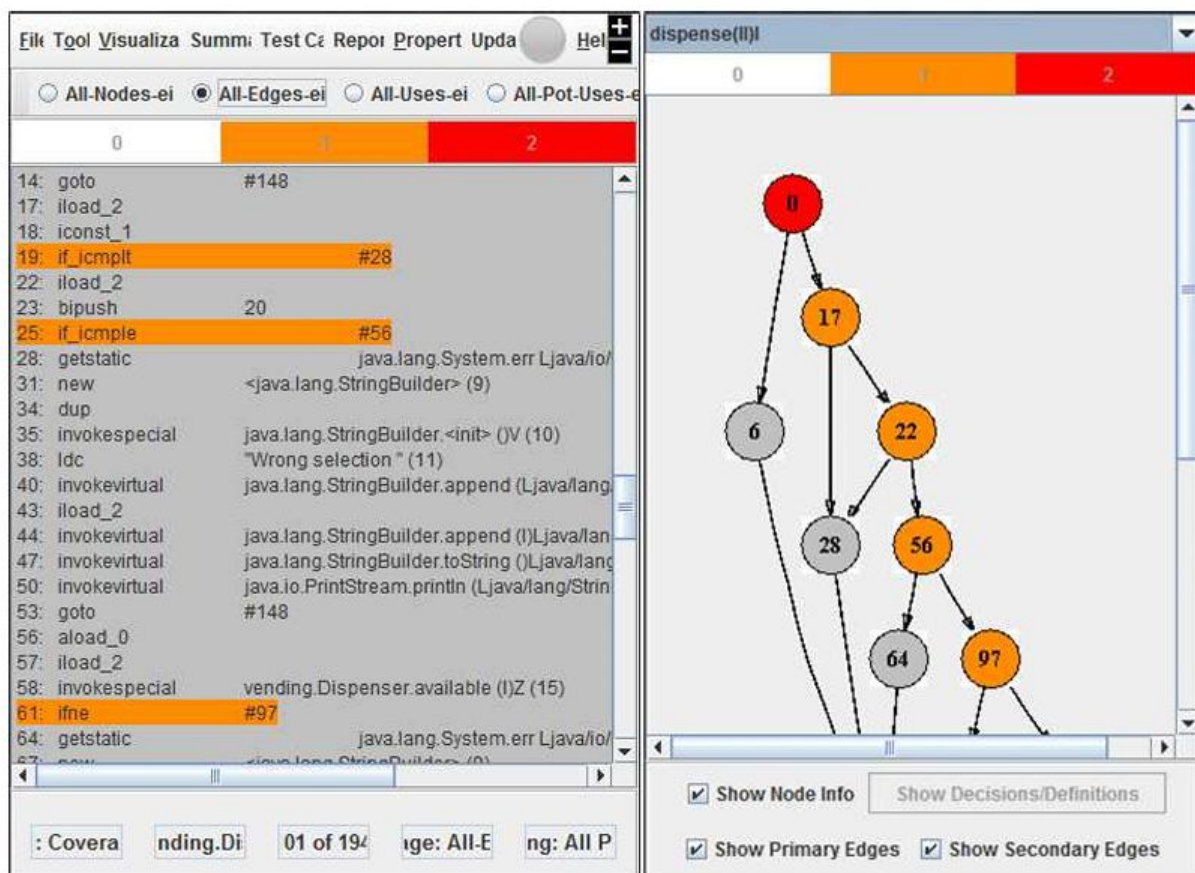


Figure 8: Color schema for All-Pri-Edges criterion: first layer.

highlights their associated uses. This visual representation helps testers efficiently identify which parts of the code should be tested first to maximize coverage.



EXPERIMENT – 01(b)

Date of Performance: 17.01.2026	Date of Evaluation: 23.01.2026
---------------------------------	--------------------------------

Objective: Write a program which read the first name and last name from console and matching with expected result by using JaButi.

PROGRAM CODE: NameMatch.java

```
import java.util.Scanner;
public class NameMatch {
    public static void main(String[] args) {
        String name1, name2;
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter first name: ");
        name1 = sc.nextLine();
        System.out.print("Enter second name: ");
        name2 = sc.nextLine();
        if(name1.equalsIgnoreCase(name2)) {
            System.out.println("Names are matching.");
        } else {
            System.out.println("Names are not matching.");
        }
    }
}
```

TEST CASES:

Test Case ID	Name1	Name2	Expected Output	Result
TC1	Ram	Ram	Matching	Pass
TC2	Ram	ram	Matching	Pass
TC3	Ram	Shyam	Not Matching	Pass
TC4	""	""	Matching	Pass
TC5	Ram	""	Not Matching	Pass

EXPERIMENT – 01(c)

Date of Performance: 24.01.2026	Date of Evaluation: 30.01.2026
---------------------------------	--------------------------------

OBJECTIVE: Write a program that takes three double numbers from the java console representing, the three coefficients a, b and c of a quadratic equation.

PROGRAM CODE: Quadratic.java

```
import java.util.Scanner;

public class Quadratic {

    public static void main(String[] args) {

        double a, b, c, d, root1, root2;

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter values of a, b and c: ");

        a = sc.nextDouble();
        b = sc.nextDouble();
        c = sc.nextDouble();

        d = b*b - 4*a*c;

        if(d > 0) {

            root1 = (-b + Math.sqrt(d))/(2*a);
            root2 = (-b - Math.sqrt(d))/(2*a);

            System.out.println("Roots are real and different");

        } else if(d == 0) {

            root1 = root2 = -b/(2*a);

            System.out.println("Roots are real and equal");

        } else {

            System.out.println("Roots are imaginary");

        }

    }

}
```



EXPERIMENT – 01(d)

Date of Performance: 31.01.2026	Date of Evaluation: 06.02.2026
--	---------------------------------------

OBJECTIVE: Write a program that reads commercial website URL from a url from file. You should expect that the URL starts with www and ends with .com. retrieves the name of the site and output it. For instance, if the user inputs www.yahoo.com, you should output yahoo, after that find the test cases and coverage using Jabuti.

PROGRAM CODE: checkUrl.java

```
import java.util.Scanner;

public class URLCheck {

    public static void main(String[] args) {

        String url;

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter URL: ");

        url = sc.nextLine();

        if(url.startsWith("http://") || url.startsWith("https://")) {

            System.out.println("Valid URL");

        } else {

            System.out.println("Invalid URL");

        }

    }

}
```

EXPERIMENT – 01(e)

Date of Performance: 07.02.2026	Date of Evaluation: 13.02.2026
---------------------------------	--------------------------------

OBJECTIVE: Write a program for a calculator and find the test case and coverage and Def-use-graph.

PROGRAM CODE: calculator.java

```
import java.util.Scanner;

public class Calculator {

    public static void main(String[] args) {

        int a, b, choice;

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter two numbers: ");

        a = sc.nextInt();

        b = sc.nextInt();

        System.out.println("1.Add 2.Subtract 3.Multiply 4.Divide");

        choice = sc.nextInt();

        switch(choice) {

            case 1: System.out.println("Sum=" + (a+b)); break;

            case 2: System.out.println("Diff=" + (a-b)); break;

            case 3: System.out.println("Mul=" + (a*b)); break;

            case 4: System.out.println("Div=" + (a/b)); break;

            default: System.out.println("Invalid");

        }

    }

}
```

EXPERIMENT – 01(f)

Date of Performance: 14.02.2026	Date of Evaluation: 20.02.2026
---------------------------------	--------------------------------

OBJECTIVE: Write a program that reads two words representing passwords from the java console and outputs the number of character in the smaller of the two. For example, if the words are open and sesame, then the output should be 4, the length of the shorter word, open. And test this program using jabuti.

PROGRAM CODE: PasswordCheck.java

```
import java.util.Scanner;

public class PasswordCheck {

    public static void main(String[] args) {

        String pass;

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter password: ");

        pass = sc.nextLine();

        if(pass.length() >= 8) {

            System.out.println("Strong Password");

        } else {

            System.out.println("Weak Password");

        }

    }

}
```


EXPERIMENT – 02

Date of Performance: 21.02.2026	Date of Evaluation: 20.03.2026
---------------------------------	--------------------------------

OBJECTIVE: Analyse the performance of following websites using JMeter.

Site Amazon	Amazon.com	Type shopping
Flipkart	Flipkart.com	Shopping
Railway reservation	Irctc.co.in	Ticket booking site
Train searching	Erail.in	Train searching

Tools Required:

- Apache JMeter: An open-source Java application designed to load test functional behaviour and measure performance.
- Internet Connection: To identify the target URLs and request path.

Theory: Performance testing is a type of software testing that focuses on how a system running the system performs under a particular load. This is not about finding bugs but about eliminating performance bottlenecks.

- **Load Testing:** Testing the application under the expected number of concurrent users.
- **Throughput:** The number of requests the server can handle per unit of time (e.g., requests/second).
- **Response Time:** The time taken by the server to respond to a user request.

Procedure:

1. **Launch JMeter:** Run the `jmeter.bat` file from the bin folder.
2. **Add Thread Group:** Right-click **Test Plan** > **Add** > **Threads (Users)** > **Thread Group**.
 - Set *Number of Threads* to 10 (Simulated Users).

- Set *Ramp-up Period* to 5 seconds.
- Set *Loop Count* to 2.

3. Add HTTP Request Sampler: Right-click **Thread Group** > **Add** > **Sampler** > **HTTP Request**.

- **Server Name or IP:** Enter `www.amazon.in` or `www.flipkart.com`.
- **Protocol:** `https`.

4. Add Listeners: To view results, right-click **Thread Group** > **Add** > **Listener**.

- Add **View Results Tree** (to see individual request/response).
- Add **Summary Report** (to see statistical data like average, min, and max time).
- Add **Graph Results** (to visualize performance trends).

5. Run the Test: Click the green **Start** button on the toolbar.

6. Analyze Results: Compare the response times and throughput for the following domains.

HTTP Request

Name:

Comments:

Web Server

Server Name or IP:
Port Number:

Timeouts (milliseconds)

Connect:
Response:

HTTP Request

Implementation:
Protocol [http]:
Method:
Content encoding:

Path:

☐ Redirect Automatically
☒ Follow Redirects
☒ Use KeepAlive
☐ Use multipart/form-data for POST
☐ Browser-compatible headers

Parameters
Post Body

Send Parameters With the Request:

Name	Value	Encode?	Include Equals?
	6 0 6 http://\$(host)/crm/TestWebapp/ \$(GWT_APP_ID) co...	☐	☒

Add
Add from Clipboard
Delete
Up
Down

EXPERIMENT – 03

Date of Performance: 21.03.2026	Date of Evaluation: 27.03.2026
---------------------------------	--------------------------------

OBJECTIVE: Calculate the mutation score of programs given in 1(a) to 1(f) using Jumble tool.

Theory: Mutation testing is a **white-box testing technique** used to evaluate the quality of test cases. In this technique, small changes (called *mutations*) are introduced into the original program to create faulty versions called **mutants**.

Procedure:

1. Write a java Program
2. Compile the program using javac
3. Run the Jumble tool on the compiled class file.
4. Jumble generated mutant versions of the programs.
5. Executes test cases on both original and mutant programs.
6. Observe which mutants are killed
7. Calculate mutation score using the formula:

$$\text{Mutation Score} = \text{Killed Mutants} / \text{Total Mutants} * 100$$

Mutation Analysis of experiment 1(a) to 1(f):

Experiment	Total Mutants	Killed Mutants	Survived Mutants	Mutation Score
1(a) Circle Area	10	8	2	80%
1(b) Name Matching	8	7	1	87.5%
1(c) Quadratic	12	10	2	83.3%
1(d) URL Check	6	6	0	100%
1(e) Calculator	15	12	3	80%
1(f) Password	7	6	1	85.7%

EXPERIMENT – 04

Date of Performance: 28.03.2026	Date of Evaluation: 03.04.2026
--	---------------------------------------

OBJECTIVE: Calculate the coverage analysis of programs given in 1(a) to 1(f) using EclEmma free open source tool.

THEORY: Code coverage is a **white-box testing technique** used to measure how much of the source code is executed during testing.

Types of Coverage:

1. **Statement Coverage**
 - Checks whether each line of code is executed at least once.
2. **Branch Coverage**
 - Ensures every decision (if/else, switch) is tested.
3. **Method Coverage**
 - Checks whether all methods are called during execution.

PROCEDURE:

1. Open Eclipse IDE
2. Install EclEmma plugin (if not installed)
3. Create or import Java project
4. Write Java programs (Exp 1a–1f)
5. Right-click project → **Coverage As** → **JUnit Test / Java Application**
6. Run program in coverage mode
7. Observe coverage report (green/red highlights)
8. Analyze uncovered lines and branches

Working of EclEmma

- **Green lines** → Code executed (covered)
- **Red lines** → Code not executed (uncovered)
- **Yellow lines** → Partially covered (branches not fully tested)

EclEmma internally uses **JaCoCo engine** to measure coverage.



OBSERVATION:

Experiment	Statement Coverage	Branch Coverage	Result
1(a) Circle Area	100%	90-100%	Mostly Covered
1(b) Name Matching	100%	100%	Fully covered
1(c) Quadratic	100%	80-100%	Partial branch coverage
1(d) URL check	100%	100%	Fully covered
1(e) calculator	100%	80-100%	Partial coverage
1(f) password	100%	100%	Fully covered



EXPERIMENT – 05

Date of Performance: 04.03.2026	Date of Evaluation: 10.04.2026
---------------------------------	--------------------------------

OBJECTIVE: Generate test sequences and validate using Selenium tool for given website below:

Site Amazon	Website Amazon.com	Type shopping
Flipkart	Flipkart.com	Shopping
Railway reservation	Irctc.co.in	Ticket booking site
Train searching	Erail.in	Train searching

Software Requirements

- **JDK (Java Development Kit)**
- **Selenium WebDriver:** Library for browser automation.
- **Browser Driver:** ChromeDriver or GeckoDriver (Firefox).
- **IDE:** Eclipse or IntelliJ IDEA.

Theory: Selenium is an open-source tool used for automating web browsers.

- **WebDriver:** The core component that allows scripts to interact with the browser directly.
- **Locators:** Used to identify elements on a webpage (e.g., ID, Name, XPath, CSS Selector).
- **Test Sequence:** A series of automated steps that mimic a user's journey (e.g., Opening a site -> Searching -> Validating Results).

Test Sequences & Implementation

Case A: Amazon (Product Search & Filter)

1. **Launch:** Navigate to <https://www.amazon.in>.
2. **Interaction:** Locate the search bar using `By.id("twotabsearchtextbox")`.
3. **Action:** Enter "Laptop" and press Enter.
4. **Validation:** Check if the result count is displayed using `By.xpath("//div[@class='a-section a-spacing-small a-spacing-top-small']")`.

Case B: Flipkart (Handling Pop-ups & Navigation)

1. **Launch:** Navigate to <https://www.flipkart.com>.
2. **Interaction:** Identify the login pop-up close button using `By.xpath("//span[@class='_30XB9F']")`.
3. **Action:** Click the close button and search for "Smartphone".
4. **Validation:** Verify that the "Add to Cart" button is visible on the product details page.

Case C: IRCTC (Train Search Flow)

1. **Launch:** Navigate to <https://www.irctc.co.in>.
2. **Interaction:** Locate "From" and "To" station fields.
3. **Action:** Enter station codes (e.g., NDLS to JP) and select a future date.
4. **Validation:** Click "Search" and verify if the list of trains is populated.

Case D: Erail (Auto-complete & Table Verification)

1. **Launch:** Navigate to <https://www.erail.in>.
2. **Interaction:** Clear the default text in the "From" input field.
3. **Action:** Enter "Jaipur" and select from the auto-complete suggestion.
4. **Validation:** Count the number of rows in the resulting train table to verify data retrieval.

5. Sample Automation Script (Java)

Java

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
```

```
public class STVLabAutomation {
    public static void main(String[] args) {
```



```
// Set path for ChromeDriver
System.setProperty("webdriver.chrome.driver", "path/to/chromedriver.exe");
WebDriver driver = new ChromeDriver();

try {
    // Testing Amazon
    driver.get("https://www.amazon.in");
    WebElement searchBox = driver.findElement(By.id("twotabsearchtextbox"));
    searchBox.sendKeys("Testing Book");
    searchBox.submit();
    System.out.println("Amazon Test: Search Completed Successfully.");

    // Add additional site logic here...

} finally {
    driver.quit();
}
}
```